Heart Disease — Monitoring (Prometheus + Grafana)

Goal: observe the live FastAPI service with HTTP-level metrics (auto-instrumented) **plus** ML-specific metrics (prediction counts, probability distribution, latency, errors) scraped by Prometheus and visualised in Grafana.

Sections:

1. Custom Prometheus metrics in `api/metrics.py`
2. Prometheus scrape configuration
3. Local stack — `docker-compose.yml`
4. Grafana dashboard — panels & PromQL queries
5. Run commands & sample `/metrics` output

In [1]:
```
import sys, json
from pathlib import Path
import yaml

ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
sys.path.insert(0, str(ROOT))
MON = ROOT / 'monitoring'
print('Monitoring dir :', MON)
```

```
Monitoring dir : C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps\Assignment1\monitoring
```

1. Custom Prometheus metrics — `api/metrics.py`

Four ML-specific metrics are registered alongside the HTTP metrics produced by `prometheus_fastapi_instrumentator`:

- **`heart_predictions_total`** (Counter) — predictions served, labelled by predicted class + model version → enables class-balance drift checks.
- **`heart_prediction_probability`** (Histogram, 0.0–1.0) — lets you spot model confidence shifting over time.
- **`heart_prediction_latency_seconds`** (Histogram) — for SLO alerts (p95 latency).
- **`heart_prediction_errors_total`** (Counter, by `error_type`) — failed predictions excluding 4xx validation errors.

In [2]:
```
from api import metrics as m
for obj_name in ['PREDICTIONS_TOTAL', 'PREDICTION_PROBABILITY',
                 'PREDICTION_LATENCY', 'PREDICTION_ERRORS']:
    obj = getattr(m, obj_name)
    print(f'{obj_name}')
```

```
    print(f'  name        : {obj._name}')
    print(f'  type        : {type(obj).__name__}')
    print(f'  labels      : {list(obj._labelnames)}')
    print(f'  description : {obj._documentation}')
    print()
```

```
PREDICTIONS_TOTAL
  name        : heart_predictions
  type        : Counter
  labels      : ['label', 'model_version']
  description : Total predictions served, labelled by predicted class.

PREDICTION_PROBABILITY
  name        : heart_prediction_probability
  type        : Histogram
  labels      : ['model_version']
  description : Distribution of predicted P(disease) values.

PREDICTION_LATENCY
  name        : heart_prediction_latency_seconds
  type        : Histogram
  labels      : ['model_version']
  description : End-to-end latency of /predict (model.predict + serialization).

PREDICTION_ERRORS
  name        : heart_prediction_errors
  type        : Counter
  labels      : ['model_version', 'error_type']
  description : Failed predictions (model errors, not validation 4xx).
```

2. Prometheus scrape configuration

Two scrape jobs:

1. `heart-disease-api` — pulls `/metrics` from the API service every 15 s.
2. `prometheus` — Prometheus self-monitors so its own scrape duration / TSDB ingest rate is queryable.

In [3]:
```
prom_cfg = yaml.safe_load((MON / 'prometheus.yml').read_text())
print(f"scrape_interval     : {prom_cfg['global']['scrape_interval']}")
print(f"external_labels     : {prom_cfg['global']['external_labels']}")
print()
for job in prom_cfg['scrape_configs']:
    targets = job['static_configs'][0]['targets']
    extra_labels = job['static_configs'][0].get('labels', {})
    print(f"job: {job['job_name']:>20s} -> {targets} {extra_labels}")
```

```
scrape_interval     : 15s
external_labels     : {'cluster': 'local-dev', 'project': 'heart-disease-mlops'}

job:    heart-disease-api -> ['api:8000'] {'service': 'heart-disease-api', 'env': 'l
ocal'}
job:           prometheus -> ['localhost:9090'] {}
```

In [4]:
```
print((MON / 'prometheus.yml').read_text())
```

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: local-dev
    project: heart-disease-mlops

scrape_configs:
  - job_name: heart-disease-api
    metrics_path: /metrics
    static_configs:
      - targets:
          - api:8000
        labels:
          service: heart-disease-api
          env: local

  - job_name: prometheus
    static_configs:
      - targets:
          - localhost:9090
```

3. Local stack — `docker-compose.yml`

Three services (api, prometheus, grafana) wired together with a private network and named volumes for TSDB + dashboard persistence.

In [5]:
```
compose = yaml.safe_load((MON / 'docker-compose.yml').read_text())
import pandas as pd
rows = []
for name, svc in compose['services'].items():
    rows.append({
        'service'   : name,
        'image'     : svc.get('image', '<built>'),
        'ports'     : ', '.join(svc.get('ports', [])),
        'depends_on': ', '.join(svc.get('depends_on', [])),
    })
pd.DataFrame(rows)
```

Out[5]:

	service	image	ports	depends_on
0	api	heart-disease-api:latest	8000:8000	
1	prometheus	prom/prometheus:v2.54.1	9090:9090	api
2	grafana	grafana/grafana:11.2.0	3000:3000	prometheus

4. Grafana dashboard — panels & PromQL queries

Dashboard JSON is mounted read-only into the Grafana container via the provisioning folder, so it is restored on every restart.

In [6]:
```
dash = json.loads((MON / 'grafana' / 'dashboards' /
                   'heart-disease-api.json').read_text())
print(f"Dashboard : {dash['title']}")
print(f"UID       : {dash['uid']}")
print(f"Refresh   : {dash['refresh']}")
print(f"Panels    : {len(dash['panels'])}")
```

```
Dashboard : Heart Disease API
UID       : heart-disease-api
Refresh   : 10s
Panels    : 10
```

In [7]:
```
rows = []
for p in dash['panels']:
    for t in p.get('targets', []):
        rows.append({
            'panel': p['title'],
            'type' : p['type'],
            'expr' : t.get('expr', '')[:80],
        })
pd.DataFrame(rows)
```

Out[7]:

	panel	type	expr
0	Predictions / sec	stat	sum(rate(heart_predictions_total[1m]))
1	API up	stat	up{job="heart-disease-api"}
2	Error rate (5xx)	stat	sum(rate(http_requests_total{status=~"5.."}[5m...
3	P95 latency	stat	histogram_quantile(0.95, sum by (le) (rate(hea...
4	Request rate by endpoint	timeseries	sum by (handler) (rate(http_requests_total{job...
5	Latency percentiles (predict)	timeseries	histogram_quantile(0.50, sum by (le) (rate(hea...
6	Latency percentiles (predict)	timeseries	histogram_quantile(0.90, sum by (le) (rate(hea...
7	Latency percentiles (predict)	timeseries	histogram_quantile(0.99, sum by (le) (rate(hea...
8	Predictions by class	timeseries	sum by (label) (rate(heart_predictions_total[1...
9	Predicted P(disease) distribution	heatmap	sum by (le) (rate(heart_prediction_probability...
10	HTTP status codes	timeseries	sum by (status) (rate(http_requests_total{job=...
11	Prediction errors	timeseries	sum by (error_type) (rate(heart_prediction_err...

5. Run commands & sample `/metrics` output

```
cd monitoring
docker compose up --build -d

# verify the API exposes metrics
curl http://localhost:8000/metrics | head -20

# generate some traffic so the dashboards have data
for i in $(seq 1 20); do \
  curl -s -X POST http://localhost:8000/predict \
       -H 'Content-Type: application/json' \
       -d @../scripts/sample_request.json > /dev/null; \
done

# open dashboards
open http://localhost:9090   # Prometheus -> Status -> Targets
open http://localhost:3000   # Grafana    -> 'Heart Disease API'
```

Reference excerpt of the `/metrics` endpoint after a few predictions:

```
# HELP heart_predictions_total Total predictions served, labelled
by predicted class.
# TYPE heart_predictions_total counter
heart_predictions_total{label="0",model_version="v1.0.0"} 14.0
heart_predictions_total{label="1",model_version="v1.0.0"} 6.0

# HELP heart_prediction_probability Distribution of predicted
P(disease) values.
# TYPE heart_prediction_probability histogram
heart_prediction_probability_bucket{model_version="v1.0.0",le="0.1"}
14.0
heart_prediction_probability_bucket{model_version="v1.0.0",le="0.5"}
14.0
heart_prediction_probability_bucket{model_version="v1.0.0",le="+Inf"}
20.0

# HELP heart_prediction_latency_seconds End-to-end latency of
/predict.
# TYPE heart_prediction_latency_seconds histogram
heart_prediction_latency_seconds_bucket{model_version="v1.0.0",le="0.025"}
18.0
heart_prediction_latency_seconds_bucket{model_version="v1.0.0",le="+Inf"}
20.0
```

What the dashboard surfaces:

- **Predictions / sec** — `sum(rate(heart_predictions_total[1m]))`
- **API up** — `up{job="heart-disease-api"}` (Prometheus health probe)
- **Latency p95** — `histogram_quantile(0.95, rate(...latency_seconds_bucket[5m]))`
- **Predicted-class distribution** — split by `label` for class-balance drift
- **Probability histogram** — to spot model confidence shifting